

Can Quantum Break ZUC? Only with a Million Qubits and a Billion Years to Spare

Anik Basu Bhaumik¹, Suman Dutta⁵, Kyungbae Jang^{1,2}, Anubhab Baksi³,
Siyi Wang¹, Amit Saha⁴, Hwajeong Seo², and Anupam Chattopadhyay¹

¹ College of Computing and Data Science, Nanyang Technological University,
Singapore

² Division of IT Convergence Engineering, Hansung University, Seoul, South Korea

³ Elektro- och informationsteknik, Lunds Universitets, Lund, Sweden

⁴ Institut National de Recherche en Informatique et en Automatique (Inria), Paris,
France & École Normale Supérieure (ENS), PSL University, Paris, France

⁵ Applied Statistics Unit, Indian Statistical Institute, Kolkata, India

Abstract. The ZUC stream cipher is integral to modern mobile communication standards, including 4G and 5G, providing secure data transmission across global networks. Recently, Dutta et al. (Indocrypt, 2024) presented the first quantum resource estimation of ZUC under Grover’s search. Although preliminary, this work marks the beginning of quantum security analysis for ZUC. In this paper, we present an improved quantum resource estimation for ZUC, offering tighter bounds for Grover-based exhaustive key search. Beyond traditional quantum resource estimations, we also provide a concrete timescale required to execute such attacks using the specified quantum resources. Our findings show that a full-round, low depth implementation of ZUC-128 can be realized with a maximum of 375 ancilla qubits, a T-count of 106536, and a T-depth of 15816. Furthermore, the Grover-based key search can be performed most efficiently using 1201 logical qubits, 170681 T gates, and a T-depth of 78189, resulting in a runtime of 1.78×10^{11} years, an improvement of 93.43% over the estimated 2.71×10^{12} years by the implementation given by Dutta et al., we also provide akin analysis for ZUC-256 with an 99.23% decrease in time. These estimations are done assuming state-of-the-art superconducting qubit error-correcting technology.

Keywords: Fault-tolerant quantum computing · Grover’s algorithm · Quantum circuit optimisation · Resource Estimation · Surface code · Space-depth trade-offs · ZUC stream cipher ·

1 Introduction

With the advent of quantum computing, cryptographic standards face an imminent threat from quantum-based attacks, making ZUC—an integral cipher in modern mobile communication standards—a critical subject of study. The ZUC

cipher is currently deployed heavily for communication of 5G and 4G communications as a part of its contribution in the 3GPP security standards (128-EEA3 and 128-EIA3). Every modern mobile device or smaller IoT devices connected via LTE relies on this encryption algorithm. Since its conception in 2010, various classical cryptanalytic techniques have been applied to ZUC [29,44,46], with a comprehensive overview provided in [33]. However, with the development of quantum computing, there has been a fundamental change in the landscape of cryptanalysis. While public-key schemes are most vulnerable—due to Shor’s efficient factorisation [39] algorithm—several quantum attacks have also been theoretically demonstrated against symmetric-key ciphers using Grover’s and Simon’s algorithms or their combinations. Hence a quantum cryptanalysis on ZUC is imminent due to its immediate implications to global security. Very recently, a preliminary study on the quantum cryptanalysis of ZUC has been conducted by Dutta et al. [13], which is extensively used in this work as a groundwork study.

To the best of our knowledge, the importance of such estimations was first emphasized by Grassl et al. [21] in their resource analysis for Grover-based attacks on AES. This work was subsequently followed-up by a series of studies on the quantum security analysis of symmetric-key cryptography and the associated resource optimisation, [3,6,22,23,24,25,28,30,38,48] to name a few. These analyses typically focus on resource metrics such as qubit count, gate count, and circuit depth to gauge scalability on real-world quantum hardware. Continuing on this trend, in this work, we focus on another commonly used cipher. We improve upon the earlier work [13] by leveraging logic synthesis tools that help us optimise for the resource metrics mentioned before, thereby enabling a proper execution of a full round of the cipher.

Execution on actual quantum hardware requires a significant level of optimisations on the logical circuit designed into a circuit using only a few specific gate sets, known as universal gate sets—analogous to netlists in classical HDL workflows, implying logical gates as the Toffoli gate to be decomposed into compatible gate sets. One of the most popular universal gate-set is the Clifford + T gate set, discussed in Section 2. We further explore the trade-offs between space and depth using the techniques proposed by Dutta et al. [14]. When estimating circuit depth (including T-depth), we assume that gates acting on disjoint qubit sets can be executed in parallel, contributing to depth of 1.

Once these circuits are decomposed to the said gate-sets we are able to measure on the actual metrics that matter for execution in the quantum hardware, like T-depth and T-count. Quantum depth reflects the execution time, while the number of qubits corresponds to memory requirements. In execution of large circuits such as that of a full round of ZUC in real-quantum hardware, there is requirement of a continuous error correcting mechanism called the surface code [18]. This error correcting flow introduces significant overhead in both the number of physical qubits required and the overall runtime. Therefore to estimate for a practical and feasible attack on ZUC it is also necessary to analyse the runtime of the entire cipher and the total resources required for the flow.

Organisation and Contributions

We discuss about the prerequisites in Section 2. The main contributions of this paper are summarised as follows:

- Section 3 is our first contributory section which provides an improved quantum resource estimation for the core components of the ZUC cipher. This includes a brief overview of the synthesis tools employed, along with key heuristics used to optimize intermediate Boolean circuit representations.
- The next section (Section 4) details the quantum resource requirements in terms of qubit count, T-count, and T-depth, based on advanced Clifford+T decompositions of both standard (2-controlled) and multi-controlled Toffoli gates. These gates are used in constructing the non-linear components of ZUC and Grover’s oracle. Additionally, we present a comparative analysis of different synthesis strategies (shallow, compact, and optimal) to highlight trade-offs among various resource metrics.
- Section 5 combines the estimated quantum resources (qubits, T-count, T-depth, and total depth) with quantum error-correcting codes to derive practical hardware requirements and attack times. Benchmarking against current superconducting qubit platforms shows substantial runtime reductions: 93.43% for ZUC-128 and 99.52% for ZUC-256 compared to prior estimates.

Finally, in Section 6, we conclude the paper with a brief summary of our results and future research directions. We also evaluate the updated MAXDEPTH criterion from NIST [34] for both the T-decomposed logical circuit (A.6) and the fault-tolerant circuit.

2 Preliminaries

We begin with a preliminary discussion of key quantum computing, logic synthesis, and quantum error correction concepts that will be used throughout this paper. We move from basic quantum computing to decomposition and reduction techniques for complex quantum gates and the role synthesis plays in these decompositions, to finally understanding how logical circuits are mapped to physical circuits and the role of error correcting codes in the scenario.

2.1 Quantum Computing Basics: Qubits and Quantum Gates

A qubit is the fundamental unit of quantum information represented as $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where $\alpha, \beta \in \mathbb{C}$ satisfying $|\alpha|^2 + |\beta|^2 = 1$. Quantum gates are represented by unitary matrices U , with inverses indicated by $U^\dagger$. Like the classical NAND and NOR gates, there are quantum gate sets that can approximate any unitary transformation on a quantum computer to an arbitrary degree of precision, known as universal gate sets. The Clifford+T gate set, rotation gates combined with the CNOT gate set, are common examples of universal gate sets.

The Clifford group is generated by three gates: Hadamard ($H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$), phase ($S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$), and CNOT ($\begin{pmatrix} I & 0 \\ 0 & X \end{pmatrix}$), where $I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ and $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$. The set is minimal, as the removal of any single gate would make it incapable of implementing certain Clifford operations. However, the Clifford gates alone do not constitute a universal gate set, as the T gate, defined as $T = \sqrt{S}$, cannot be generated solely using the Clifford operations. Consequently, the Clifford group, when augmented with the T-gate, forms a universal quantum gate set, enabling arbitrary quantum computations.

The Toffoli gate is a three-qubit gate in which the first two qubits act as control qubits, while the third (target) qubit is flipped if and only if both control qubits are in the $|1\rangle$ state. Mathematically, Toffoli : $|a, b, c\rangle \rightarrow |a, b, c \oplus ab\rangle$. Consequently, the Toffoli gate is also denoted as CCNOT (or CCX) gate.

The (2-controlled) Toffoli gate can be further extended to multi-controlled Toffoli (MCT) gates. Despite its utility, the implementation of MCT gates using the Clifford+T gate set requires careful decomposition to minimise the T-count, T-depth, and the required number of qubits. These optimisations are essential for practical quantum computation, where resource efficiency is critical. A comprehensive summary of the existing benchmarks for Clifford+T decompositions of single and multi-controlled Toffoli gates is provided in [14, Tables I, II]. For more details, one may refer to sources as [36, Chapter 3].

2.2 Design of ZUC-128/256

ZUC is a word-oriented stream cipher. ZUC-128 takes a 128-bit key, 128-bit IV, and a 240-bit constant; whereas ZUC-256 utilizes a 256-bit key, 128-bit IV, and a 112-bit constant. ZUC operates on three interconnected layers: (a) Linear Feedback Shift Register (LFSR) layer, (b) Bit-Reorganization (BR) layer, (c) Non-Linear Function (NLF) layer.

The key, IV, and constants (496 bits in total for both variants of ZUC) are loaded into the LFSR, as described in [13, Sections 2.1 and 2.2]. After the key/IV loading, both variants follow an almost identical process, allowing for a unified description of the cipher's operation.

There are two phases in the operation of ZUC: The initialisation phase and the key-stream generation phase. The schematic diagrams for the initialisation and key-stream generation are omitted from here for brevity but can be found in [42, Figures 1, 2]. The initialisation phase runs for 32 rounds (without generating the key-stream bits). The key-stream generation happens thereafter. The operational descriptions for each components are described in [42]. A small description of the operational semantics of the components can be found in the appendices A.2

2.3 Grover's Search on Stream Ciphers

Given $f : \{0, 1\}^n \rightarrow \{0, 1\}$ with the promise that $f(x_0) = 1$ for a fixed $x_0 \in \{0, 1\}^n$, and $f(x) = 0$ if $x \neq x_0$, Grover's algorithm can find $x_0 \in \{0, 1\}^n$ using $\mathcal{O}(\sqrt{2^n})$ oracle queries, whereas the classical search may require up to $\mathcal{O}(2^n)$ queries in the worst case. The algorithm relies on amplitude amplification, a

technique that increases the amplitude of the target state while suppressing the amplitudes of non-solution states, thereby enhancing search efficiency.

Given any key, $K \in \{0, 1\}^k$ and an $IV \in \{0, 1\}^m$, let $\text{ZUC}_{K,IV} = ks$ denote the key-stream generated from ZUC. For any given key stream ks of length ρ , we employ Grover’s search for key recovery as follows. Since IV is typically public

Algorithm 1: Grover Search for Key K_0 via ZUC Oracle

- 1: Construct oracle U_f s.t. $f(K, IV) = 1$ if $\text{ZUC}_{K,IV} = ks$, else 0.
 - 2: Init k qubits: $|0\rangle^{\otimes k} \xrightarrow{H^{\otimes k}} |\mathcal{K}\rangle = \frac{1}{\sqrt{2^k}} \sum_j |K_j\rangle$.
 - 3: Apply U_f : $\frac{1}{\sqrt{2^k}} \sum_j (-1)^{f(K_j)} |K_j\rangle$.
 - 4: Apply diffusion $D = 2|\mathcal{K}\rangle\langle\mathcal{K}| - I$.
 - 5: Repeat Steps 3–4 for $\mathcal{O}(\sqrt{2^k})$ iterations.
 - 6: Measure qubits; output observed key K_0 .
-

information, the first step can be modified such that f takes only K as input and satisfies $f(K) = 1$ if ZUC_K generates the key-stream ks . Considering the scenario where two different keys, K_i and K_j , generate the same key-stream of length ρ , the collision probability, $\Pr(C_\rho(K_i) = C_\rho(K_j)) \approx \frac{2^k - 1}{2^\rho}$, can be reduced to $\mathcal{O}(2^{-c})$ by setting the key-stream length to $\rho = k + c$. We refer to the oracle used for marking as $\mathcal{ENC}$, this oracle contains the ZUC cipher implemented as a quantum circuit. The schematic diagram of the corresponding Grover’s oracle is shown in [3, Fig. 1].

2.4 Logic Synthesis and Its Applications

In the logic synthesis phase, Register Transfer Level (RTL) descriptions are translated into gate-level netlists supported by specific technology libraries. The primary optimization metrics in this process are the per-gate cost and the total number of gates, which together determine the area of the resulting circuit. In the quantum domain, a similar optimization problem arises, aimed at minimizing the logical depth and the number of qubits, which correspond to execution time and hardware area, respectively. Notably, reducing circuit depth often incurs an exponential increase in the number of required qubits. Therefore, a critical trade-off must be achieved between circuit depth and the ancilla qubits used. In Section 3, we explore a range of tools like the EPFL Logic synthesis libraries [41] and ABC [9] designed to effectively manage this trade-off between depth and ancilla.

2.5 Reduction of Multi-controlled Toffoli into Toffoli gates

Efficient decomposition of multi-controlled Toffoli (MCT) gates into the Clifford+T gate set, via Toffoli-based constructions, has seen significant advancements in recent years. The techniques used have been described below.

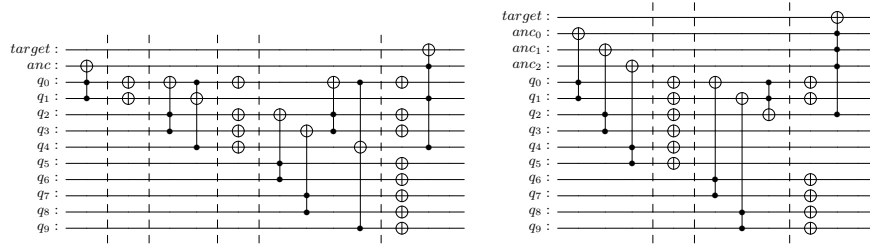
Optimal Toffoli-depth. As demonstrated in [14], an n -controlled Toffoli gate can be decomposed into $n-1$ basic Toffoli gates using $n-2$ additional ancilla

qubits, achieving an optimal Toffoli-depth of $\lceil \log_2 n \rceil$. For example, [14, Fig. 12] presents a 7-MCT circuit decomposition with a Toffoli depth of $\lceil \log_2 7 \rceil = 3$, using 5 ancilla qubits, and 6 Toffoli gates.

Conditionally Clean Ancilla. In the conditionally clean ancilla technique (Khattar et al. [27]), the construction begins with at least one clean ancilla qubit. Initially, a basic Toffoli gate is applied with the clean ancilla as the target. If the ancilla is flipped, this indicates that both control qubits were in the $|1\rangle$ state. Applying Pauli-X gates to these control qubits then transforms them to the $|0\rangle$ state, making them available to be reused as ancilla qubits in subsequent rounds. The process continues iteratively until all control qubits are exhausted. Once the relevant information from all control qubits is accumulated into a smaller set, a final (smaller) MCT gate is applied. This gate uses the accumulated qubits and the original ancilla qubits as controls and the original target qubit as the final target. Note that, if the initial clean ancilla remains unmodified in the first step, the target qubit remains unaffected.

Using this method, an n -controlled Toffoli (MCT) gate can be decomposed into a circuit with $2n - 3$ Toffoli gates where the Toffoli-depth is bounded by $4 \log_2 n$. In Figure 1a, we illustrate an 8-controlled Toffoli decomposition using two ancilla qubits, achieving a Toffoli-depth of 6.

Trade-offs between Clean Ancilla and Toffoli Depth. In a recent work [14], the conditionally clean ancilla technique has been extended to scenarios where additional clean ancilla qubits are available at the outset, resulting in a precise trade-off formulation between the number of clean ancilla used and the Toffoli-depth of the resulting circuit decomposition. Under constraints on available ancilla, this method yields the minimum Toffoli-depth achievable using conditionally clean ancilla. An example of the decomposition of a 10-controlled Toffoli (10-MCT) gate using additional clean ancilla is illustrated in Figure 1b.



(a) Conditionally clean ancilla technique (based on [27]). (b) Conditionally clean ancilla with space-depth trade-offs (based on [14]).

Fig. 1: 10-MCT circuit decompositions following implementation strategies.

2.6 Reduction of Toffoli into T gates

Toffoli gates are further reduced to T-Gates, such that the entire circuit can be modelled on the Clifford + T gate set. To do so, we employ two decomposition

techniques, Amy et al. [1] proposed a decomposition with T-depth of 4 but one less Clifford gate, with overall circuit depth 8, they also propose a decomposition with T-Depth of 3 and overall depth of 9 as shown in Figure 2a which is employed in our decomposition techniques. Gidney et al. [19] gives a measurement-based uncomputation strategy for T decomposition with a T-depth of 1 and overall depth of 8 as shown in Figure 2b.

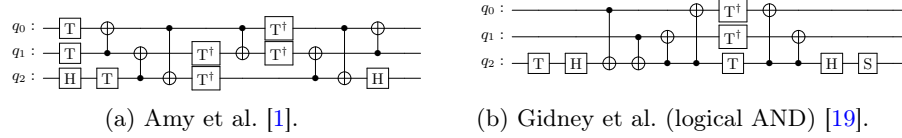


Fig. 2: Toffoli $\rightarrow$ Clifford+T decomposition methods.

2.7 Quantum Error Correction

Physical qubits are noisy and short-lived unlike logical qubits, building dependable representations of logical qubits with given physical qubits requires Quantum Error Correction (QEC), fixing errors made during idle storage and gate operations. A stronger QEC scheme is required for longer and larger algorithms like Grover's search in ZUC, increasing requirement in time and resources. The surface code [18] is considered to be the most promising model in application to real quantum hardware for error correction models. A 2D array of nearest-neighbor physical qubits is used by the surface code to detect and correct errors. One logical qubit in an $n \times n$ array of physical qubits ($n = 2d - 1$) is encoded by a distance- d surface code. Each cycle 1-2-qubit gates, qubit measurements, and standard error correction mechanisms (required to monitor errors to prevent logical failures). The technique in [16] assumes a mesh of classical computing units and an $L \times L$ grid of logical qubits to accomplish constant-time error detection rounds. Hence, the physical qubits and quantum gates estimated for a surface-code-based quantum computer give a good approximation of the resources needed for running large-scale circuits like those for ZUC.

The surface code cannot directly realize non-Clifford gates such as the T gate, which are essential for universal quantum computation. These gates are implemented using magic state distillation [8], this protocol repeatedly prepares and purifies noisy ancilla states to achieve the high fidelity needed for fault-tolerant execution. Magic state distillation is resource-intensive, demanding dedicated factories and many qubits per state, and becomes a key bottleneck for T-gate-heavy algorithms like Grover's search.

3 Efficient Quantum Synthesis of ZUC

As outlined in Section 2, ZUC comprises three primary layers: the Linear Feedback Shift Register (LFSR) layer, the Bit Reorganization layer, and the Non-Linear

Function (NLF) layer. The NLF layer includes both a linear component and S-box operations. To enable an efficient quantum implementation of the NLF layer, we employ reversible circuit synthesis techniques, as detailed below.

3.1 Synthesis of Linear Layers

There are two linear layers L_1 and L_2 in the ZUC cipher, each can be represented by a 32×32 binary matrix, they are related by $L_1 = L_2^T$. To optimally synthesise these matrices we use various tools and convert them into lists of XOR expressions. These are subsequently translated into quantum representations by mapping each XOR operation to a corresponding CNOT gate. The methodologies and outcomes of these tools are as follows.

Summary of Algorithms in Linear Layer Synthesis. Dutta et al. [13] present a two-step CNOT-based synthesis using the technique from [35], achieving depth 46 and 202 CNOT gates for $m = 10$ (see [13, Table 3]) by triangularizing, transposing, and grouping matrices to minimize depth beyond Gauss–Jordan elimination. Xiang et al. [45] propose an s-XOR approach for efficient 3232 matrix decomposition, outperforming Boyar–Peralta and Paar algorithms for AES and ZUC (Table 1). Baksi et al. [5] extend the Boyar–Peralta tool to higher fan-in XOR gates for efficient XOR3 implementations and further enhance Xiang’s in-place synthesis for larger matrices [4]. Yuan et al. [47] generalize this framework to optimize XOR count or depth, achieving state-of-the-art AES results and the lowest-depth ZUC synthesis with minimal qubit overhead. A comprehensive summary of various synthesis tools with the corresponding resource requirements generated for the ZUC linear layers L_1 and L_2 are presented in Table 1.

Table 1: Synthesis results of ZUC linear layers.

Method	L_1			L_2		
	Depth	#CNOT	#Qubits	Depth	#CNOT	#Qubits
Patel et al. [35]	46	202	32	46	202	32
Baksi et al. [5]	13	150	87	13	150	87
Xiang et al. [45]	25	84	32	19	84	32
Baksi et al. [4]	19	182	32	19	182	32
Yuan et al. [47]	10	89	32	10	88	32

3.2 Synthesis of S-boxes

Various classical optimisation techniques for S-boxes exist in the literature. In our work, we leverage classical algorithms to optimise S-box logic and apply appropriate mapping strategies to synthesise efficient quantum circuit representations. During our optimization process of the S-box, we followed certain synthesis flows, described as follows.

The **EPFL logic synthesis** toolkit provides a suite of tools for classical logic optimization and quantum circuit synthesis. The process begins with *Lorina*, which parses classical logic representations (e.g., given in PLA, Verilog formats) into internal formats such as XOR-AND graphs (XAGs) or k -LUT graphs.

These graphs are then processed using *Mockturtle*, a logic network manipulation library. Subsequently, *Caterpillar* maps the optimized logic networks into quantum logic using various strategy algorithms [41] (see Fig. 3). The mapped logic is converted into executable Python code using the *Tweedledum* library, facilitating simulation and integration into larger quantum algorithms.

ABC is a classical logic synthesis and optimization tool [9] that operates on various representations, such as AIGs (AND-Inverter Graphs). In our experiments, we use ABC to simplify both raw logic (e.g., truth tables) and pre-optimized representations (e.g., from [26]) encoded in bench format. The resulting logic is mapped to k -LUTs, which are subsequently synthesized into a quantum circuit using the EPFL flow as shown in Figure 4.

DORCIS [4,10] and **LIGHTER-R** [12], which are in-place synthesis algorithms particularly suitable for small permutation functions (typically up to 4-bit S-boxes). We also utilize **Tweedledum** which supports implementation of certain synthesis algorithms [32] for Boolean functions and permutations, including those relevant to S-box design.

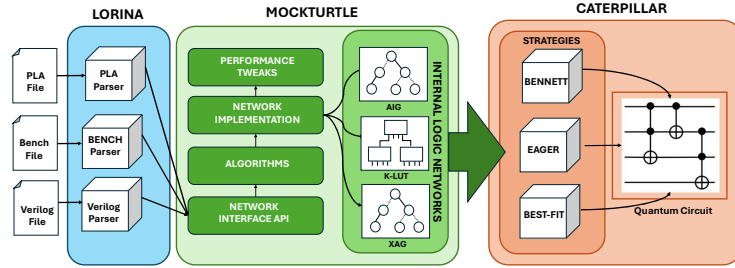


Fig. 3: EPFL logic synthesis schematic.

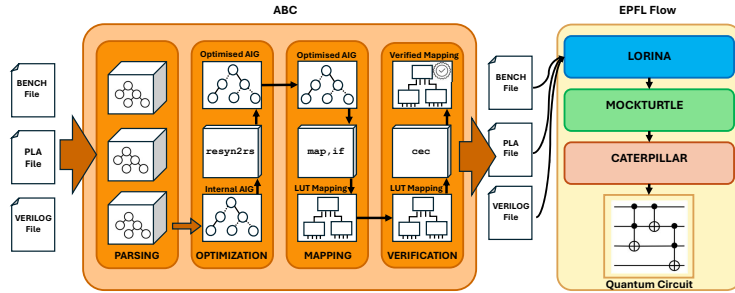


Fig. 4: Synthesis schematic of quantum circuits with ABC.

S_0 as Feistel Structure. The S-box S_0 can be represented as a composition of three distinct 4-bit mappings (which can be interpreted as S-boxes), as described in [15, Chapter 10.3.1], namely P_1 : 9F0EFF2A040C7539, P_2 : 8D6570C4B1EAF392 and P_3 : 26A60DAF33D509CD. Among these, P_2 is a bijective mapping, for which we employed in-place synthesis techniques. For the non-bijective mappings P_1 and P_3 , we utilised mapping-based synthesis tools. The overall structure is illus-

trated in [15, Fig 10.1]. A similar diagram has also been added in the appendices for reference A.3

Synthesis of P_1 and P_3 : As observed, both P_1 and P_3 are non-bijective. In-place synthesis tools such as DORCIS, LIGHTER-R, and the TBS algorithm from the Tweedledum library are not capable of handling non-bijective mappings. Consequently, we resorted to out-of-place synthesis methods. In particular, we employed synthesis flows based on the ABC frontend and the EPFL logic synthesis toolkit for these mappings.

Synthesis of P_2 : Given that P_2 is bijective, both in-place and out-of-place synthesis techniques are applicable. In-place synthesis methods are usually preferred because of their lower ancillary requirements. We provide two quantum circuit implementations across these configurations:

- (i) An 8-qubit implementation with P_2 in-place (Figure 5a).
- (ii) A 16-qubit implementation with P_2 in-place (Figure 5b).

Table 2: Comparison of metrics for various flows for ZUC S_0 mappings.

#Qubits	P_1 flow	P_2 flow	P_3 flow	NCT-depth	#3-MCT	#Toffoli	#CNOT	#NOT
8	EPFL	DORCIS	EPFL	65	2	26	30	15
8	EPFL	LIGHTER-R	EPFL	63	2	26	30	15
8	EPFL	TBS	EPFL	81	14	34	34	15
16	EPFL	DORCIS	EPFL	49	1	21	27	14
16	EPFL	LIGHTER-R	EPFL	48	1	21	27	14
16	EPFL	TBS	EPFL	58	7	25	29	14
16	ABC	DORCIS	ABC	47	1	21	27	4
16	ABC	LIGHTER-R	ABC	46	1	21	27	4
16	ABC	TBS	ABC	56	7	25	29	4

Table 2 shows that implementing P_2 using LIGHTER-R provides better depth compared to DORCIS or Tweedledum. In Table 2 NCT-depth refers to the depth of the circuit when it is at a level containing only NOT-CNOT-Toffoli gates. For EPFL, the best-fit mapping strategy was employed, as it provides a compact 8-qubit representation suitable for our quantum implementation. ABC was used as the frontend, and we observed an overall reduction in circuit depth. Across all mappings, the pre-optimized versions generated using ABC demonstrated improvements in both gate count and depth. The ABC commands used were: “if -K 4 -a; resyn2rs; if -K 4”. Additionally, in the EPFL back-end of the ABC flow, we also applied the best-fit strategy.

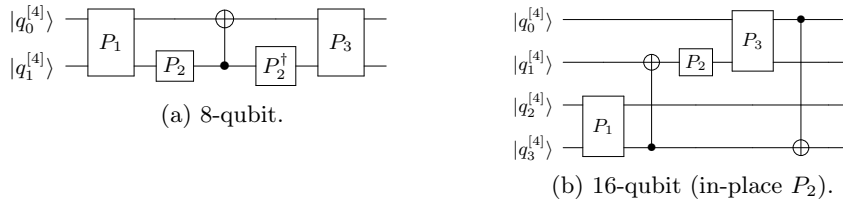


Fig. 5: Quantum Implementations for S_0 .

Various Quantum Realisations of S_1 . The S_1 S-box has highly non-linear properties, as is evident from our results in Table 3, synthesising S_1 , yields a lot of and operation implying large number of Toffoli gates. We employ classical logic synthesis techniques, for generating the quantum versions of this S-box. We apply classical optimization algorithms to various logic representations, exclusively for the S-box S_1 . Among the classical representations, we employ the XOR-AND expression lists proposed by Zou et al. [26] for different components of S_1 (available online)⁶.

Result Summary. Our results clearly demonstrate the space–depth trade-off described in [14]: reducing qubit count generally increases circuit depth, and vice versa. For example, synthesising for Zou et al.’s [26] implementation generally we require 409 qubits, but applying ABC (“resyn2rs; if -K 6 -a;”) three to four times reduces qubits at the cost of depth (Zou-ABC (6-LUT)), We also observe increasing LUT size in ABC decreases depth (Figure 4), See Table 3. This shows ABC’s effectiveness as a post-synthesis optimiser.

Using EPFL-based synthesis strategies (Section 2), we see that free-qubit allocation strategies like Bennett and Eager reduce depth, whereas the best-fit strategy, which minimises qubits, causes substantial depth growth. These results highlight the inherent spatial–temporal trade-off in quantum circuit design. Resource details for implementing S_1 are shown in Table 3.

Table 3: Quantum resource estimation of S_1 following multiple synthesis flows.

Flow	#Qubits	NCT-Depth	#X	#CX	#CCX	#n-MCT				
						3	4	5	6	7
Tweedledum (TBS) [32]	8	939	4	37	125	226	250	206	79	15
Tweedledum (TBS-Bi)	8	755	4	30	97	155	241	163	56	13
Tweedledum (TBS-Multi) [40]	8	654	3	18	80	144	206	132	70	9
EPFL (Best-fit) [31]	16	1037	12	32	116	225	292	216	118	32
EPFL (Bennett) [7]	1220	154	20	3604	2400	0	0	0	0	0
EPFL (Eager)	472	414	16	3600	2394	0	0	0	0	0
ABC-EPFL (Best-fit)	16	1036	4	32	116	225	292	216	118	32
ABC-EPFL (Bennett)	963	1386	714	2304	1902	0	0	0	0	0
ABC-EPFL (Eager)	262	1445	714	2304	1902	0	0	0	0	0
Zou et al. [26]	160	103	14	218	34	0	0	0	0	0
Zou-ABC (4-LUT)	81	205	60	294	137	22	4	0	0	0
Zou-ABC (5-LUT)	68	209	50	245	169	22	4	0	0	0
Zou-ABC (6-LUT)	62	252	42	226	222	12	0	0	0	0

4 Improved Quantum Resource Estimation for ZUC

In this section, we present the improved quantum resource estimations for ZUC-128 and ZUC-256. As discussed earlier, S-box construction can involve multi-controlled Toffoli gates. However, such gates are not must be decomposed into

⁶ <https://github.com/xsm222/classical-circuit-implementation-of-ZUC-S1-box/tree/master>

a universal gate-set (See 2.1). To accurately estimate the required resources, we first decompose the multi-controlled Toffoli gates into standard Toffoli and Clifford gates using the strategies outlined in Section 2.5. Subsequently, we apply the Toffoli $\rightarrow$ T gate decomposition techniques (see Section 2.6) to obtain the final resource metrics in terms of Clifford + T gates.

For MCT decomposition, we use the strategy of Khattar et al. [27], which employs conditionally clean ancilla qubits. Specifically, for the S_0 , we consider the in-place 8-bit result labeled EPFL – DORCIS – EPFL from Table 2, and for S_1 , we use the Zou – ABC (6-LUT) synthesis result from Table 3. The linear layer implementation follows Baksi et al. [5], as stated in Table 1.

4.1 Quantum Resource Estimation for LFSR

We propose a parallelization strategy that enables the LFSR operations from consecutive rounds to be executed concurrently, improving from the ripple carry adder implementation [11] as implemented in [13]. In particular, the LFSR computation for a given round can be initiated in parallel with that of the preceding round, thereby improving circuit-level efficiency. Consider the LFSR computation at any given round, which performs five modular additions (see Section 2.2): $s_{16} = 2^{15}s_{15} + 2^{17}s_{13} + 2^{21}s_{10} + 2^{20}s_4 + (1 + 2^8)s_0$.

The input to the next round is a right-shifted version of the current state, i.e., $(s_1, s_2, \dots, s_{15}, s_{16})$. Hence, most of the additions for the next LFSR round can be executed in parallel with the current round, with the exception of the term involving s_{16} , which depends on the current computation: $s_{17} = 2^{15}s_{16} + 2^{17}s_{14} + 2^{21}s_{11} + 2^{20}s_5 + (1 + 2^8)s_1$. As a result of this parallelization, only the remaining addition involving $2^{15}s_{16}$ needs to be performed in the next round⁷. This strategy generalizes to subsequent rounds as well.

To enable this level of parallelism, we assign six ancilla qubits to allow for three parallel quantum adders. Empirically, we observe that three such instances are sufficient, as each quantum adder can be reused after three rounds. For example, the adder used in round i is reused in round $(i + 3)$. Table 4 presents a comparison of the quantum resources required for the LFSR between our implementation and that of Dutta et al. [13]. In our design, except for the initial round ($i = 1$), the depth in each round i accounts only for the final addition of $2^{15}s_{14+i}$, which contributes to the quantum circuit depth by 134.

Table 4: Comparison of the quantum resources for ZUC LFSR.

Source/Reference	Round	#Qubits	#X	#CX	#CCX	NCT-Depth
Dutta et al. [13]	Each	31 + 2	348	893	364	337
This Work	Initial	31 + 6	348	893	364	337
	Subsequent					134

⁷ More precisely, a final modulo- 2^{31} addition is performed to incorporate any carry.

4.2 Quantum Resource Estimation for NLF Layer

The NLF layer comprises two main components: the linear layers (L_1 and L_2) and the S-box S_0 and S_1). Structurally, each linear layer is connected to four S-boxes. This motivates the need to estimate the resource requirements for both S-box and linear layer implementations, as detailed in Section 3.

We consider two distinct implementations for S_0 and S_1 with minimal ancilla requirements: an out-of-place implementation of S_1 (Zou – ABC(5-LUT)), and an in-place implementation of S_0 (EPFL – DORCIS - EPFL), chosen for its significantly lower circuit depth and zero ancilla usage. As evident from Table 5, the maximum size of the MCT gate in the S-box construction is $n = 4$. Applying the conditionally clean ancilla method [27], each 3-MCT and 4-MCT gate requires one additional ancilla qubit. Specifically, each 4-MCT is implemented using 5 Toffoli gates and 4 NOT (X) gates, whereas each 3-MCT uses 3 Toffoli gates and does not require any additional X gates. Each MCT decomposition incurs an additional depth of $(2n-3)$. The total Toffoli-count for implementing S_0 is given by: $26 + 0 \times 5 + 2 \times 3 = 32$, and that of S_1 is $169 + 4 \times 5 + 22 \times 3 = 255$. The corresponding Toffoli depth after decomposition (using [27]) for S_0 is $65 + (2 \times 3 - 3) \times 2 = 71$, and that of S_1 is $209 + (2 \times 4 - 3) \times 4 + (2 \times 3 - 3) \times 22 = 295$. The total NOT gate count for S_1 is $4 \times 4 + 50 = 66$. NCT-Depth in all tables henceforth signifies the depth of NOT, CNOT, and Toffoli gates together in the circuit.

Table 5: Quantum resource estimation for ZUC S-boxes.

S-box	Ancilla	NCT-depth	#X	#CX	#CCX
S_0	$0 + 1$	71	15	30	32
S_1	$60 + 1$	295	66	245	255
S	122	295	162	550	574

From [13, Sec. 4.4], implementing in-place operations $W_1 = R_1 \boxplus X_1$ requires $2 \times (31-2) = 58$ NOT gates, $5 \times (31+1) - 7 = 153$ CNOT gates, $2 \times (31+1) - 3 = 61$ Toffoli gates, and a depth of $2 \times (31+1) + 2 = 66$, and implementing $W_2 = R_2 \oplus X_2$ requires 32 CNOT gates to map the 16 least significant bits of s_7 and the 16 most significant bits of s_5 .

Therefore, total quantum resource required for the NLF layer, assuming the linear layer design from [4] (182 CNOT gates, circuit depth 19), updating R_1 and R_2 require a total of $(58 + 162) = 220$ NOT gate, $(153 + 32 + 2 \times 182 + 550) = 1099$ CNOT gate, and $(61 + 574) = 635$ Toffoli gates, with a circuit depth of $(66 + 1 + 19 + 295) = 381$.

4.3 Quantum Resource Estimation at Various Stages

Here, we provide a detailed estimation of quantum resource requirement at different stages of ZUC, depending on its internal architecture and execution sequence.

Key/IV Loading. The LFSR comprises 16 registers, each of 31 bits. Assuming that one qubit is allocated per bit, the total qubit requirement is $31 \times 16 = 496$

qubits. The Key, Initialization Vector (IV), and the constant d_i are embedded within this register space. For ZUC-128, loading the key requires 128 CNOT gates, while the IV requires a maximum of 128 NOT gates. Similarly, the constant d_i requires up to 240 NOT gates. For ZUC-256, loading the key requires 256 CNOT gates, and loading the IV and the constants requires 240 NOT gates.

ZUC Initialisation. Prior to key-stream generation, the ZUC circuit must undergo 32 rounds of initialisation. After updating R_1 and R_2 , one needs to compute $W = (X_0 \oplus R_1) \boxplus R_2$. Storing W requires 32 working qubits, where we copy X_0 using 32 CNOT gates. The XOR operation $(X_0 \oplus R_1)$ involves another 32 CNOT gates. For the in-place modular addition with R_2 , the operation requires one ancilla qubit, $2(31 - 2) = 58$ NOT gates, $5(32 - 7) = 153$ CNOT gates, and $2 \times 31 - 1 = 61$ Toffoli gates, having a circuit depth of $2 \times 32 + 2 = 66$, as per the design in [11, Table 1]. Subsequently, the rightmost bit of W is discarded, and the remaining bits are added to the LFSR module using two ancilla qubits. This process requires: $2(31 - 2) = 58$ NOT gates, $5(31 + 1) = 156$ CNOT gates, and $2 \times 31 - 3 = 61$ Toffoli gates, with a circuit depth of $2 \times 31 + 6 = 68$ (following [11, Table 1.]). Hence, the total resources required: $(58 + 58) = 116$ NOT gates $(32 + 32 + 153 + 156) = 373$ CNOT gates $(61 + 61) = 122$ Toffoli gates and a depth of $(1 + 1 + 66 + 68) = 136$.

ZUC Key-stream Generation. In Key-stream generation, after computing $W = (X_0 \oplus R_1) \boxplus R_2$, instead of discarding the last qubit and adding it to the LFSR, X_3 is XOR-ed with W using 32 CNOT gates, and produces the key-stream $Z = X_3 \oplus W$. Therefore, total the resource required: 58 NOT gates, $32 + 32 + 153 + 32 = 249$ CNOT gates, 61 Toffoli gates, and a circuit depth of $1 + 1 + 66 + 1 = 69$.

Consequently, the resource requirements for one complete round of ZUC initialization can be determined by adding the resources needed for the key/IV loading, LFSR update, NLF layer, and the generation of W in the post-NLF layer. One complete round of ZUC-128/256 initialisation requires $(496 + 31 + 64 + 120 + 32) = 743$ working qubits, $(368/240 + 348 + 220 + 116) = 1052/924$ NOT gates, $(128/256 + 893 + 1099 + 373) = 2493/2621$ CNOT gates, and $(364 + 635 + 122) = 1121$ Toffoli gates, with the maximum circuit depth of $(1 + 337 + 381 + 136) = 855$ for the first round and $(1 + 134 + 381 + 136) = 652$ for the subsequent rounds.

Similarly, one complete round of ZUC-128/256 key-stream generation requires $(348 + 220 + 58) = 626$ NOT gates, $(893 + 1099 + 249) = 2241$ CNOT gates, and $(364 + 635 + 61) = 1060$ Toffoli gates, with the maximum circuit depth of $(134 + 381 + 69) = 584$.

4.4 Total Quantum Resources for $\mathcal{ENC}$

Let us now estimate the total quantum resources required to implement the ZUC cipher ($\mathcal{ENC}$), as illustrated in Figure 6, with the circuit decomposed into Toffoli gates. It is essential to consider that ZUC undergoes 32 rounds of initialization

prior to the key stream (key-stream) generation. From Section 4.3, the initialization of ZUC-128/256 requires 29568/33664 NOT gates, 79776/83872 CNOT gates, and a total of 35872 Toffoli gates.

Once the key-stream generation begins, each round produces a 32-bit key. To prevent collision in the exhaustive search, we assume that at least 5 and 9 rounds are necessary for ZUC-128 and ZUC-256, respectively; we denote this as $k = 5/9$. The number of NOT gates required during key stream generation is 3130/5634; the number of CNOT gates is 11205/20169; and the number of Toffoli gates is 5300/9540. The overall circuit depth is computed as $(855 + 652 \times 31 + k \times 584) = 23987/26323$, corresponding to ZUC-128 and ZUC-256, respectively. A summary of the total resources required is provided in Table 6.

Table 6: Gate counts and circuit depth for ZUC-128 and ZUC-256.

Variant	#X	#CX	#CCX	NCT-depth
ZUC-128	36794	90981	41172	23987
ZUC-256	35202	104041	45412	26323

4.5 Comparison of ZUC $\mathcal{ENC}$ Variants Based on Resource Metrics

In this section, we compare different versions of the ZUC cipher based on the selection of synthesized variants of its components, as outlined in Table 10. We adopt a straightforward strategy to guide our selection: Identify one variant optimized for depth and another for space (i.e., qubit count). For the depth-optimized (shallow) architecture, we select two linear layer implementations with the lowest depth, along with two implementations of each S-box namely $S_0 \rightarrow S_{00}, S_{01}$ and $S_1 \rightarrow S_{10}, S_{11}$. We select three configurations, Shallow, Compact and Optimal. **Shallow** focuses on minimizing the circuit depth, hence uses Bakshi et al. (13 depth, 87q) for L_1/L_2 (See Table 1), EPFL-LIGHTER-R-EPFL and ABC-LIGHTER-R-ABC for S_0 (See Table 2), and Zou et al. with EPFL (Bennett) for S_1 (See Table 3). **Compact** focuses on a minimum size circuit hence uses Yuan et al. (10 depth) for L_1/L_2 (See Table 1), EPFL-LIGHTER-R-EPFL (63q) and EPFL-TBS-EPFL (81q) for S_0 (See Table 2), and TBS-Multi and TBS-Bi for S_1 (See Table 3). **Optimal** versions aim for a balanced implementation hence it reuses Yuan et al. (10 depth) (See Table 1) and the Compact S_0 choices, with Zou-ABC (5-LUT) and Zou-ABC (6-LUT) for S_1 (See Table: 3).

4.6 Resource Estimation of Grover Circuit for ZUC

The total resource requirements for single iteration of the Grover circuit are estimated using the formula: $2 \times \mathcal{ENC}_{res} + \mathcal{MCT}_{res}$, where $\mathcal{MCT}_{res}$ denotes the resource cost of the n -MCT used in amplitude amplification. The resources for $\mathcal{ENC}$ is summed twice for the state marking phase, namely for $\mathcal{ENC}$ and $\mathcal{ENC}^\dagger$. We utilize the decomposition strategy from Dutta et al. [14], which leverages at least six reusable ancilla qubits from the parallel LFSR structure (Section 3). It requires only five ancillas ($m_1 = 3$, $m_2 = 2$) and reduces depth without additional qubit overhead.

From the configurations in sub-section 4.5, the total resource requirements for a complete Grover iteration is calculated. Given that parallel Toffoli gates contribute only once to the circuit depth, we assume that the Toffoli depth scales linearly with circuit depth. Thus, to obtain the quantum depth, we multiply the circuit depth in Toffoli level to the total depth, as given by Amy et al. [1] and Gidney et al. [19], which is 9. The data for comparison with different parameters and techniques for each of these versions is available in A.5 .

Resources for n -MCT in Grover’s Search. In Grover’s search algorithm, the MCT gate plays a crucial role, responsible for flipping the phase of the target state. The key-stream length for ZUC-128/256 is 160 and 288 bits, respectively. Therefore, we consider n -MCT constructions for $n = 160$ and $n = 288$.

From Section 2.5, there are several decomposition strategies for converting n -MCT gates into standard (2-controlled) Toffoli gates. These are subsequently decomposed into T gates following Section 2.6. The total number of Toffoli gates required for implementing an n -MCT is $2n - 3$, resulting in Toffoli counts of 317 and 573 for ZUC-128 and ZUC-256, respectively. The Dutta et al. (optimal) [14] decomposition yields Toffoli depths of $\lceil \log_2(160) \rceil = 8$ and $\lceil \log_2(288) \rceil = 9$. In contrast, the conditionally clean ancilla method by Khattar et al. [27] results in depths of 317 and 573, respectively. Additionally, we apply the depth-reduction method proposed in [14], where the ancilla configuration is chosen as $m_1 = 2$ and $m_2 = 3$. A detailed estimation for all the combinations can be found in A.4

Resources for ZUC $\mathcal{ENC}$ Variants. *Shallow version* results in the lowest circuit depth (18082), lowest qubit count (881), and the most efficient gate count among the variants for ZUC-256. The corresponding Toffoli count obtained is 26634, implying the T-count is $26634 \times 7 = 186438$ (from Amy et al. [1]), or the T-count is $26634 \times 4 = 106536$ using Gidney et al. [19]. Similarly, the corresponding resource requirement for ZUC-128 are: 32132 NOT gates, 81879 CNOT gates, 24226 Toffoli gates, which implies, a T count of $24226 \times 7 = 169582$ (from Amy et al. [1]) and a T count of $24226 \times 4 = 96904$ (from Gidney et al. [19]). Further, the circuit depth becomes 16890 with a qubit count of 881. Since the S-box S_1 requires an additional 8 ancilla, we include 320 extra ancilla qubits, bringing the total qubit count to $881 + 320 = 1201$.

Compact version achieves the lowest circuit depth (168,388) using only 563 qubits, and contains an overall minimal gate count, we adopt the n -MCT reduction strategy by [27], to receive a Toffoli-count of 286492, implying a T count of $286492 \times 7 = 2005444$ (from [1]) and a T count of $286492 \times 4 = 1145968$ (from [19]) for ZUC-256. In a similar setup, ZUC-128 requires 202332 NOT gates, 67301 CNOT gates, 258732 Toffoli gates, which implies a T count of $258732 \times 7 = 1811124$ (Amy et al. [1]) and a T count of $258732 \times 4 = 1034928$ (Gidney et al. [19]). Furthermore, the circuit depth becomes 152532 with a qubit count of 563. No additional ancilla is required, as all NLF components are implemented in-place.

Optimal version adopts the optimal decomposition from [14]. The resource profile for ZUC-256 is as follows. full depth: 24560, qubit count: 685, Toffoli

count: 45412, which implies T-count: 317884 (Amy et al. [1]), or 181648 (Gidney et al. [19]). For ZUC-128 with the same configuration requires 35610 NOT gates, 84099 CNOT gates, 41172 Toffoli gates, which implies 288204 (Amy et al.), or 164688 (Gidney et al.) T gates, with a circuit depth of 22736 using 685 qubits. Here, we have an additional 120 ancilla qubits required for the S block, the final qubit count becomes: $685 + 120 = 805$.

Estimation of T-depth. The T gates in this circuit are primarily generated through the decomposition of Toffoli gates. The main sources of Toffoli gates in ZUC arise from the LFSR, the NLF, and the n -MCT gates used in the amplification stage. We compute the T-depth of the LFSR based on its structure: six sequential quantum additions in the initial round, followed by two additions in each subsequent round. According to Toffoli decompositions provided by Amy et al. [1] and Gidney et al. [19]

Each quantum addition (for LFSR) contributes a T-depth of 244 and 32, respectively. Using the decomposition by Amy et al. [1]. The total T-depth for the LFSR is $6 \times 244 = 1464$ for the initial round, and $2 \times 244 = 488$ for subsequent rounds. Similarly, using the decomposition by Gidney et al. [19], the total T-depth for the LFSR is: 192 for the initial round, and 64 for subsequent rounds.

For the NLF, we use the Toffoli-depth of the selected configurations, as previously discussed. The cipher requires $k = 5/9$ rounds of key-stream generation, along with 32 rounds of initialization. The total T-depth of ZUC, therefore, can be expressed as: $\text{T-depth}_{\text{ZUC}} = \text{T-depth}_{\text{LFSR}(\text{init.})} + (31 + k) \text{T-depth}_{\text{LFSR}(\text{sub.})} + (32 + k) \text{T-depth}_{\text{NLF}}$. For the **shallow version**, with $\text{T-depth}_{\text{NLF}} = 180$, the $\text{T-depth}_{\text{ZUC}} = 39012/43124$ (Amy et al. [1]) and $\text{T-depth}_{\text{ZUC}} = 15816/17512$ (Gidney et al. [19]) for $k = 5/9$. In the **compact version**, where $\text{T-depth}_{\text{NLF}} = 3846$, $\text{T-depth}_{\text{ZUC}} = 445938/494042$ and $\text{T-depth}_{\text{ZUC}} = 287100/318124$ under the respective decompositions. Finally, the **optimal version** with $\text{T-depth}_{\text{NLF}} = 338$ yields $\text{T-depth}_{\text{ZUC}} = 56550/62558$ and $\text{T-depth}_{\text{ZUC}} = 27508/30468$. The final T-depth values, including contributions from the amplification stage as per the decomposition from [14], are summarized in Table 7.

5 Runtime Estimation on Fault-Tolerant Architectures

The most resource-intensive task of fault-tolerant surface codes is implementing the T gate, which relies on a faulty magic state that must be purified via magic state distillation. The circuits responsible for state purification demand significant logical qubit resources. In this section, we estimate the additional overhead required for executing ZUC under this paradigm.

Let T_U^c denote the T-count of a circuit U , and T_U^d its T-depth. The T-width is defined as $T^w U = T_U^c / T_U^d$, representing the average number of T gates that can be executed in parallel per depth layer. Each T gate requires a logical magic state $|A_L\rangle := \frac{|0_L\rangle + e^{i\pi/4}|1_L\rangle}{\sqrt{2}}$. For a successful execution of U , these states must have an error rate no greater than $p_{\text{out}} = 1/T_U^c$.

We adopt the Bravyi-Kitaev 15-to-1 distillation scheme [8] and follow the modified methodology proposed by Amy et al. [2], which begins with a magic

Table 7: Resource comparison of various versions for one iteration of ZUC.

Version	Variant	Toffoli decomposition	#Qubits	#T	T-depth	Full-depth
Shallow	ZUC-128	Amy et al. [1]	1201	170681	78189	304515
	ZUC-128	Gidney et al. [19]	1201	97532	31742	304515
	ZUC-256	Amy et al. [1]	1201	188433	86467	326133
	ZUC-256	Gidney et al. [19]	1201	107676	35170	326133
Compact	ZUC-128	Amy et al. [1]	563	1812223	892041	2746071
	ZUC-128	Gidney et al. [19]	563	1035556	574310	2746071
	ZUC-256	Amy et al. [1]	563	2007439	988303	3031641
	ZUC-256	Gidney et al. [19]	563	1147108	636394	3031641
Optimal	ZUC-128	Amy et al. [1]	805	289303	113265	409743
	ZUC-128	Gidney et al. [19]	805	165316	55126	409743
	ZUC-256	Amy et al. [1]	805	319879	125335	442737
	ZUC-256	Gidney et al. [19]	805	182788	61082	442737

state of error rate p_{in} , injects it into an error-correcting code, and distills it to a lower error rate. If the resulting error still exceeds p_{out} , additional rounds of distillation are applied, exponentially decreasing the error. Assuming negligible logical errors during distillation (valid for sufficiently high code distances), the ideal output error after one round is $p_{\text{dist}} = 35p_{\text{in}}^3$. Following [18], we assume that the error rate of surface code gate p_g is approximately one-tenth of p_{in} , i.e., $p_g = p_{\text{in}}/10$, since about 10 gates must function correctly before error protection becomes effective.

To account for logical errors introduced during distillation, we introduce a parameter ε such that $\varepsilon p_{\text{dist}}$ quantifies this contribution, giving $p_{\text{out}} = (1 + \varepsilon)p_{\text{dist}}$. In a balanced scenario where $\varepsilon = 1$, the logical circuit introduces as much error as is eliminated by distillation. Fowler et al. [17] provides a framework for estimating the number of distillation rounds required to achieve a target output error rate, along with the minimum code distances necessary at each stage. This process is summarized in Algorithm 2; where d_1 denotes the code distance for the top distillation layer, d_2 for the next, and so on.

Algorithm 2: Estimating number of rounds of magic state distillation and corresponding distances of concatenated codes (based on [2]).

1: Input: $\varepsilon, p_{\text{in}}, p_{\text{out}}, p_g (= p_{\text{in}}/10)$	
2: $d \leftarrow$ empty list $[]$	8: Find minimum d_i such that
3: $p \leftarrow p_{\text{out}}$	$192d_i(100p_g)^{\frac{d_i+1}{2}} < \frac{\varepsilon p_i}{1+\varepsilon}$
4: $i \leftarrow 0$	9: $p \leftarrow \sqrt[3]{p_i/(35(1+\varepsilon))}$
5: repeat	10: $d.\text{append}(d_i)$
6: $i \leftarrow i + 1$	11: until $p > p_{\text{in}}$
7: $p_i \leftarrow p$	12: Output: $d = [d_1, \dots, d_i]$

We consider the shallow version of ZUC-128 (with the Toffoli decomposition by Gidney et al. [19]) as a representative example for quantum resource estimation. The T -count of our ZUC-128 circuit is denoted as $T_{\text{ZUC-128}}^c$ (see Table 7) for this shallow implementation. The T -count for the complete Grover-based exhaustive key search algorithm (hereafter referred to as GA) is given by $T_{GA}^c = \lfloor \pi/4 \times 2^{64} \rfloor \times T_{\text{ZUC-128}}^c$. The corresponding target output error rate for magic state distillation is $p_{\text{out}} = 1/T_{GA}^c$. Assuming a magic state injection error rate of $p_{\text{in}} = 0.02$, a per-gate error rate of $p_g = 10^{-3}$, and setting $\varepsilon = 1$, Algorithm 2 recommends a three-layer magic state distillation scheme with code distances $d_1 = 21$, $d_2 = 10$, and $d_3 = 7$.

The bottom distillation layer generates a single $|A_L\rangle$ magic state through three distillation layers requires $N_{\text{dist}} = 16 \times 15 \times 15 = 3600$ input states. These are encoded using a distance- d_3 surface code, which requires approximately $2.5 \times 1.25 \times d_3^2$ physical qubits per logical qubit, each round of distillation requires $10d_3 = 70$ surface code cycles. The middle layer uses a surface code with distance $d_2 = 10$, requiring approximately $2.5 \times 1.25 \times d_2^2$ physical qubits per logical qubit. The total number of physical qubits required in this layer is approximately $16 \times 15 \times 2.5 \times 1.25 \times d_2^2$, and the distillation round completes in $10d_2 = 100$ surface code cycles. The top distillation layer employs a distance- $d_1 = 21$ surface code, with each logical qubit occupying about $2.5 \times 1.25 \times d_1^2$ physical qubits. The total qubit requirement at this layer is $16 \times 2.5 \times 1.25 \times d_1^2$, and the distillation takes $10d_1 = 210$ surface code cycles to complete.

It is important to note that the physical qubits in the bottom distillation layer can be reused in the subsequent layers. Thus, the maximum physical qubits required to successfully distill a single purified state $|A_L\rangle$ is approximately equal to that required by the bottom layer. The full concatenated distillation process spans a total of $\sigma_{\text{dist}} = 70 + 100 + 210 = 380$ surface code cycles.

The T-depth and total depth of the full Grover's algorithm the total number of surface code cycles (σ_{GA}) is calculated in Table 8. T-gates dominate most of the computational runtime, as the qubits involved in Clifford operations remain idle for most of the computation. Hence, the overall runtime is primarily dictated by the time-consuming generation of magic states.

Table 8 focuses on the ZUC variants decomposed by Gidney et al. [19], which demonstrate lower T-depth and T-count relative to other known constructions. Among these, the shallow variant stands out for requiring the least total number of surface code cycles to execute the entire algorithm.

The space-depth trade-off costs for various ZUC quantum circuit designs from 8 using NIST's MAXDEPTH [34] model are shown in Table 9. It demonstrates that while compact circuits suffer significantly from large depth even though they use fewer qubits, shallow circuits achieve the lowest costs because their smaller depth keeps the squared-depth cost manageable. Under the depth-dominated cost metric, optimal designs are in the middle but still trail Shallow.

The duration of a surface code cycle varies across quantum hardware platforms. For superconducting qubits, such as those employed by IBM and Google, it typically ranges from 200 ns to 1 μ s, depending on factors like code distance

Table 8: Resource analysis of various versions of $\mathcal{ENC}$ using Grover’s search with surface code.

Version	Variant	Code distances	#Physical qubits	Total cycles
Shallow	ZUC-128	[21, 10, 7]	$2^{22.9}$	2^{82}
	ZUC-256	[34, 15, 9]	$2^{24.18}$	$2^{146.3}$
Compact	ZUC-128	[22, 11, 8]	$2^{22.3}$	$2^{85.3}$
	ZUC-256	[34, 15, 9]	$2^{23.8}$	$2^{149.5}$
Optimal	ZUC-128	[21, 10, 7]	$2^{22.51}$	$2^{82.6}$
	ZUC-256	[34, 15, 9]	$2^{24.07}$	$2^{146.7}$

Table 9: Trade-off metrics for decomposed ZUC quantum circuits for fault tolerance analysis considering NIST’s MAXDEPTH [34].

Version	Variant	$Td - M$	$Td^2 - M$
Shallow	ZUC-128	1.87×2^{104}	1.87×2^{186}
	ZUC-256	1.39×2^{170}	1.72×2^{316}
Compact	ZUC-128	1.52×2^{107}	1.87×2^{192}
	ZUC-256	1.23×2^{173}	1.74×2^{322}
Optimal	ZUC-128	1.08×2^{105}	1.64×2^{187}
	ZUC-256	1.71×2^{170}	1.39×2^{317}

and control precision [19]. In contrast, trapped-ion systems operate more slowly, with cycle times generally between 10 μ s and 1 ms, owing to longer gate and measurement durations [43]. In Table 10, we estimate the total runtime of our algorithm assuming a superconducting qubit platform. Furthermore, we compare our results with the current state-of-the-art implementation of ZUC [13], highlighting the improved runtime achieved through a more efficient circuit design.

Table 10: Run-time estimation on Grover’s search on ZUC as per latest (in 2025) superconducting qubit technology.

Variant	#Physical qubits	Time (years)*	Time (years) †
ZUC-128 [13]	$2^{23.5}$	5.41×10^{11}	2.71×10^{12}
ZUC-128 [This work]	$2^{22.9}$	3.56×10^{10}	1.78×10^{11}
ZUC-256 [13]	$2^{24.9}$	1.46×10^{32}	7.29×10^{32}
ZUC-256 [This work]	$2^{24.18}$	7.04×10^{29}	3.52×10^{30}

* : 200ns for 1-surface code cycle, † : 1 μ s for 1-surface code cycle.

6 Conclusion

In this paper, we demonstrated that synthesis and optimization flows enable practical quantum circuit design for the ZUC stream cipher. Our techniques achieve a significant trade-off between qubit count and circuit depth, reducing execution cycles by $2^{2.7}$ times and the physical qubit requirement by over 1.85 mil-

lion in the ZUC-128 implementation. Compared to the currently best-known results, our approach yields a 93.43% and 99.52% reduction in runtime for ZUC-128 and ZUC-256, respectively. Additionally, the lowest T-depth and the circuit depth have been reduced to 1.03×2^{26} and 1.00×2^{28} , respectively; with the optimal designs maintaining low depth while outperforming compact versions. As an idea for future works, we aim to refine these estimates even further using the advanced distillation methods such as Litinski’s protocol, magic state cultivation and use other error correcting methods such as the yoke surface code as provided in [20].

References

1. Amy, M., Maslov, D., Mosca, M., Roetteler, M.: A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits. *Trans. Comp.-Aided Des. Integ. Cir. Sys.* **32**(6), 818–830 (2013). <https://doi.org/10.1109/TCAD.2013.2244643> 7, 16, 17, 18, 26, 29
2. Amy, M., Matteo, O.D., Gheorghiu, V., Mosca, M., Parent, A., Schanck, J.: Estimating the cost of generic quantum pre-image attacks on SHA-2 and SHA-3 (2017). https://doi.org/10.1007/978-3-319-69453-5_18 17, 18
3. Anand, R., Maitra, A., Maitra, S., Mukherjee, C.S., Mukhopadhyay, S.: Quantum resource estimation for FSR based symmetric ciphers and related Grover’s attacks. In: *Progress in Cryptology – INDOCRYPT 2021*. pp. 179–198. Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-92518-5_9 2, 5, 24
4. Baksi, A., Chakraborty, S., Chattopadhyay, A., Chun, M., Islam, S.H., Jang, K., Kim, H., Oh, Y., Roy, S., Seo, H., Wang, S.: Quantum implementation of linear and non-linear layers. In: *IEEE 37th International System-on-Chip Conference (SOCC)*. pp. 1–6 (2024). <https://doi.org/10.1109/SOCC62300.2024.10737862> 8, 9, 13, 27
5. Baksi, A., Dasu, V.A., Karmakar, B., Chattopadhyay, A., Isobe, T.: Three input Exclusive-OR gate support for Boyar-Peralta’s algorithm (2021). https://doi.org/10.1007/978-3-030-92518-5_7 8, 12, 27
6. Bathe, B., Anand, R., Dutta, S.: Evaluation of Grover’s algorithm toward quantum cryptanalysis on ChaCha. *Quantum Inf. Process* **20** (2021). <https://doi.org/10.1007/s11128-021-03322-7> 2
7. Bennett, C.H.: Time/space trade-offs for reversible computation. *SIAM Journal on Computing* **18**(4), 766–776 (1989). <https://doi.org/10.1137/0218053> 11
8. Bravyi, S., Kitaev, A.: Universal quantum computation with ideal Clifford gates and noisy ancillas. *Phys. Rev. A* **71**, 022316 (2005). <https://doi.org/10.1103/PhysRevA.71.022316> 7, 17
9. Brayton, R., Mishchenko, A.: ABC: An academic industrial-strength verification tool. In: *Computer Aided Verification*. vol. 6174, pp. 24–40. Springer Berlin Heidelberg (2010). https://doi.org/10.1007/978-3-642-14295-6_5 5, 9
10. Chun, M., A., B., Chattopadhyay, A.: DORCIS: Depth optimized quantum implementation of substitution boxes. *Cryptology ePrint Archive* (2023), <https://eprint.iacr.org/2023/286> 9
11. Cuccaro, S.A., Draper, T.G., Kutin, S.A., Moulton, D.P.: A new quantum ripple-carry addition circuit. *arXiv* (2004), <https://arxiv.org/abs/quant-ph/0410184> 12, 14

12. Dasu, V.A., Baksi, A., Sarkar, S., Chattopadhyay, A.: LIGHTER-R: Optimized reversible circuit implementation for SBoxes. In: 32nd IEEE International System-on-Chip Conference (SOCC). pp. 260–265 (2019). <https://doi.org/10.1109/SOCC46988.2019.1570548320> 9
13. Dutta, S., Ghatak, A., Chattopadhyay, A., Maitra, S.: Quantum cryptanalysis of ZUC and related resource estimation. In: Progress in Cryptology – INDOCRYPT 2024. pp. 329–355. Springer-Verlag (2024). https://doi.org/10.1007/978-3-031-80308-6_15 2, 4, 8, 12, 13, 20
14. Dutta, S., Wang, S., Baksi, A., Chattopadhyay, A., Maitra, S.: Exact space-depth trade-offs in multicontrolled toffoli decomposition (2025). <https://doi.org/10.1103/PhysRevA.111.052611> 2, 4, 5, 6, 11, 15, 16, 17, 26, 27, 28
15. ETSI/SAGE Technical report, V.: Specification of the 3gpp confidentiality and integrity algorithms 128-EEA3 and 128-EIA3 - document 4: Design and evaluation report (2011), https://www.gsma.com/about-us/wp-content/uploads/2014/12/EEA3_EIA3_Design_Evaluation_v2_0.pdf 9, 10, 26
16. Fowler, A.G.: Minimum weight perfect matching of fault-tolerant topological quantum error correction in average $o(1)$ parallel time (2014), <https://arxiv.org/abs/1307.1740> 7
17. Fowler, A.G., Devitt, S.J., Jones, C.: Surface code implementation of block code state distillation. Scientific Reports **3**(1) (2013). <https://doi.org/10.1038/srep01939> 18
18. Fowler, A.G., Mariantoni, M., Martinis, J.M., Cleland, A.N.: Surface codes: Towards practical large-scale quantum computation. Physical Review A **86**(3) (2012). <https://doi.org/10.1103/physreva.86.032324> 2, 7, 18
19. Gidney, C.: Halving the cost of quantum addition. Quantum **2**, 74 (2018). <https://doi.org/10.22331/q-2018-06-18-74> 7, 16, 17, 18, 19, 20, 26, 29
20. Gidney, C.: How to factor 2048 bit rsa integers with less than a million noisy qubits (2025), <https://arxiv.org/abs/2505.15917> 21
21. Grassl, M., Langenberg, B., Roetteler, M., Steinwandt, R.: Applying Grover’s algorithm to AES: Quantum resource estimates. In: Post-Quantum Cryptography. pp. 29–43. Springer International Publishing, Cham (2016). https://doi.org/10.1007/978-3-319-29360-8_3 2
22. Huang, Z., Sun, S.: Synthesizing quantum circuits of AES with lower T-depth and less qubits. In: Agrawal, S., Lin, D. (eds.) Advances in Cryptology – ASIACRYPT 2022. pp. 614–644. Springer Nature Switzerland, Cham (2022). https://doi.org/10.1007/978-3-031-22969-5_21 2
23. Jang, K., Baksi, A., Kim, H., Song, G., Seo, H., Chattopadhyay, A.: Quantum analysis of AES. IACR Communications in Cryptology **2**(1) (2025). <https://doi.org/10.62056/ay11zo-3y> 2
24. Jang, K., Lim, S., Oh, Y., Kim, H., Baksi, A., Chakraborty, S., Seo, H.: Quantum implementation and analysis of SHA-2 and SHA-3. IEEE Transactions on Emerging Topics in Computing pp. 1–15 (2025). <https://doi.org/10.1109/TETC.2025.3546648> 2
25. Jaques, S., Naehrig, M., Roetteler, M., Virdia, F.: Implementing grover oracles for quantum key search on AES and LowMC (2020). https://doi.org/10.1007/978-3-030-45724-2_10 2, 28
26. Jian, Z., SiMing, X., ChangZhong, Z., WenLing, W.: An efficient quantum circuit implementation of ZUC-128 cipher with low T-depth. SCIENTIA SINICA Physica, Mechanica and Astronomica **54**(12), 120311 (2024). <https://doi.org/10.1360/SSPMA-2024-0147> 9, 11

27. Khattar, T., Gidney, C.: Rise of conditionally clean ancillae for optimizing quantum circuits (2024), <https://arxiv.org/abs/2407.17966> 6, 12, 13, 16, 26, 27, 28
28. Leander, G., May, A.: Grover meets Simon – Quantumly attacking the FX-construction. In: Takagi, T., Peyrin, T. (eds.) *Advances in Cryptology – ASIACRYPT 2017*. pp. 161–178. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-70697-9_6 2
29. Liu, F., Meier, W., Sarkar, S., Wang, G., Ito, R., Isobe, T.: New cryptanalysis of ZUC-256 initialization using modular differences. *IACR Trans. Symmetric Cryptol.* **2022**, 152–190 (2022). <https://doi.org/10.46586/tosc.v2022.i3.152-190> 2
30. Liu, Q., Preneel, B., Zhao, Z., Wang, M.: Improved quantum circuits for AES: Reducing the depth and the number of qubits. In: Guo, J., Steinfeld, R. (eds.) *Advances in Cryptology – ASIACRYPT 2023*. pp. 67–98. Springer Nature Singapore (2023). https://doi.org/10.1007/978-981-99-8727-6_3 2
31. Meuli, G., Soeken, M., Rötteler, M., Wiebe, N., Micheli, G.D.: A best-fit mapping algorithm to facilitate ESOP-decomposition in Clifford+T quantum network synthesis. 23rd Asia and South Pacific Design Automation Conference (ASP-DAC) pp. 664–669 (2018). <https://doi.org/10.1109/ASPDAC.2018.8297398> 11
32. Miller, D.M., Maslov, D., Dueck, G.W.: A transformation based algorithm for reversible logic synthesis. In: *DAC '03: Proceedings of the 40th annual Design Automation Conference*. pp. 318–323 (2003). <https://doi.org/10.1145/775832.775915> 9, 11
33. Mukherjee, C.S., Roy, D., Maitra, S.: *Design and Cryptanalysis of ZUC - a stream cipher in mobile telephony*. Springer Singapore (2021). https://doi.org/10.1007/978-981-33-4882-0_2, 25
34. NIST.: Call for additional digital signature schemes for the post-quantum cryptography standardization process (2022), <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sept-2022.pdf> 3, 19, 20
35. Patel, K.N., Markov, I.L., Hayes, J.P.: Optimal synthesis of linear reversible circuits. *Quantum Info. Comput.* **8**(3), 282–294 (2008). <https://doi.org/10.26421/QIC8.3-4-4> 8
36. Perriello, S.: Quantum circuits for information set decoding quantum cryptanalysis of code-based cryptosystems. Ph.D. thesis, Politecnico di Milano (2024), <https://www.politesi.polimi.it/retrieve/ef311c33-5427-4869-865f-94dc5ee733ed/Thesis.pdf> 4
37. Sarah, D., Peter, C.: On the practical cost of grover for AES key recovery. NIST Standardization Conference (Fifth) (2024), <https://csrc.nist.gov/csrc/media/Events/2024/fifth-pqc-standardization-conference/documents/papers/on-practical-cost-of-grover.pdf> 28
38. Shi, H., Feng, X.: Quantum circuits of AES with a low-depth linear layer and a new structure. In: Chung, K.M., Sasaki, Y. (eds.) *Advances in Cryptology – ASIACRYPT 2024*. pp. 358–395. Springer Nature Singapore (2025). https://doi.org/10.1007/978-981-96-0944-4_12 2
39. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing* **26**(5), 1484–1509 (1997). <https://doi.org/10.1137/S0097539795293172> 2
40. Soeken, M., Dueck, G.W., Rahman, M.M., Miller, D.M.: An extension of transformation-based reversible and quantum circuit synthesis. In: *2016 IEEE International Symposium on Circuits and Systems (ISCAS)*. pp. 2290–2293 (2016). <https://doi.org/10.1109/ISCAS.2016.7539041> 11

41. Soeken, M., Riener, H., Haaswijk, W., Testa, E., Schmitt, B., Meuli, G., Mozafari, F., Lee, S.Y., Calvino, A.T., Marakkalage, D.S., Micheli, G.D.: The EPFL logic synthesis libraries (2022), <https://arxiv.org/abs/1805.05121> 5, 9
42. Design team: The ZUC-256 stream cipher (2018), <http://www.is.cas.cn/ztzl2016/zouchongzhi/201801/W020180416526664982687.pdf> 4
43. Todaro, S.L., Verma, V.B., McCormick, K.C., Allcock, D.T.C., Mirin, R.P., Wineland, D.J., Nam, S.W., Wilson, A.C., Leibfried, D., Slichter, D.H.: State read-out of a trapped ion qubit using a trap-integrated superconducting photon detector. *Phys. Rev. Lett.* **126**, 010501 (2021). <https://doi.org/10.1103/PhysRevLett.126.010501> 20
44. Wu, H., Huang, T., Nguyen, P.H., Wang, H., Ling, S.: Differential attacks against stream cipher ZUC. In: *Advances in Cryptology – ASIACRYPT 2012*. vol. 7658, pp. 262–277. Springer Berlin Heidelberg (2012). https://doi.org/10.1007/978-3-642-34961-4_17 2
45. Xiang, Z., Zeng, X., Lin, D., Bao, Z., Zhang, S.: Optimizing implementations of linear layers. *IACR Trans. Symmetric Cryptol.* **2020**(2), 120–145 (2020). <https://doi.org/10.13154/tosc.v2020.i2.120-145> 8
46. Yang, J., Johansson, T., Maximov, A.: Spectral analysis of ZUC-256. *IACR Trans. Symmetric Cryptol.* **2020**, 266–288 (2020). <https://doi.org/10.13154/tosc.v2020.i1.266-288> 2
47. Yuan, Y., Wu, W., Shi, T., Zhang, L., Zhang, Y.: A framework to improve the implementations of linear layers. *IACR Trans. Symmetric Cryptol.* **2024**(2), 322–347 (2024). <https://doi.org/10.46586/tosc.v2024.i2.322-347> 8, 27, 28
48. Zou, J., Wei, Z., Sun, S., Liu, X., Wu, W.: Quantum circuit implementations of AES with fewer qubits. In: Moriai, S., Wang, H. (eds.) *Advances in Cryptology – ASIACRYPT 2020*. pp. 697–726. Springer International Publishing, Cham (2020). https://doi.org/10.1007/978-3-030-64834-3_24 2

A More Details on Synthesis and Time Estimation of ZUC

A.1 Grover Oracle for ZUC

A schematic diagram of the Grover’s oracle for an exhaustive key search of a stream cipher, is provided in Figure 6 derived from [3].

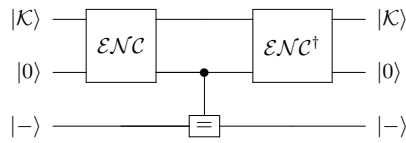


Fig. 6: Grover’s oracle for comparing all possible key-stream bits from $|K\rangle$ with the input key-stream ks .

A.2 Design of ZUC 128/256

We now provide a brief operational description of the individual layers. ZUC is based on a single LFSR in the Fibonacci configuration, i.e., the feedback occurs only at the highest-indexed state. The LFSR contains sixteen states, each a 31-bit register, with the taps defined by the primitive polynomial $f(x) = x^{16} - (2^{15}x^{15} + 2^{17}x^{13} + 2^{21}x^{10} + 2^{20}x^4 + (1 + 2^8))$ over the field $\mathbb{F}_p$, where $p = 2^{31} - 1$. Consequently, the LFSR update function is given by $\text{feedback}_f(s_0, \dots, s_{15}) = 2^{15}s_{15} + 2^{17}s_{13} + 2^{21}s_{10} + 2^{20}s_4 + (1 + 2^8)s_0$.

In the bit-reorganization (BR) layer, specific registers from the LFSR are selected and reorganized as: $X_0 = s_{15H} \| s_{14L}$; $X_1 = s_{11L} \| s_{9H}$; $X_2 = s_{7L} \| s_{5H}$; $X_3 = s_{2L} \| s_{0H}$, where s_{iH} denotes the 16 most significant bits (MSB) of s_i , and s_{iL} represents the 16 least significant bits (LSB) of s_i .

The non-linear feedback (NLF) layer takes (X_0, X_1, X_2) as input and, based on the registers R_1, R_2 (32-bit, each), generates the output $W = (X_0 \oplus R_1) \boxplus R_2$ through the intermediate computations $W_1 = R_1 \boxplus X_1$ and $W_2 = R_2 \oplus X_2$. The registers R_1 and R_2 are then updated as $R_1 = S(L_1(W_{1L} \| W_{2H}))$ and $R_2 = S(L_2(W_{2L} \| W_{1H}))$, respectively. The substitution box (S-box) S utilized within the non-linear function is defined using two components S-boxes, S_0 and S_1 , as specified in [33, Tables 3.1 and 3.2]. Note that, $S_0, S_1 : \{0, 1\}^8 \rightarrow \{0, 1\}^8$, while $S : \{0, 1\}^{32} \rightarrow \{0, 1\}^{32}$. For a 32-bit tuple (x_1, x_2, x_3, x_4) with $x_i \in \{0, 1\}^8$, S is applied as follows: $S(x_1 \| x_2 \| x_3 \| x_4) := S_0(x_1) \| S_1(x_2) \| S_0(x_3) \| S_1(x_4)$. The linear maps L_1 and L_2 used in the non-linear function are defined as: $L_1(X) = X \oplus (X \lll 2) \oplus (X \lll 10) \oplus (X \lll 18) \oplus (X \lll 24)$, $L_2(X) = X \oplus (X \lll 8) \oplus (X \lll 14) \oplus (X \lll 22) \oplus (X \lll 30)$. The only difference between the initialisation phase and the key-stream generation phase lies in the handling of W . During initialisation, the rightmost bit of W is discarded, and the remaining 31-bit value is added to the LFSR feedback. In contrast, during key-stream generation, W is XOR-ed with X_3 to produce the key-stream.

A.3 Synthesis of ZUC S-box S_0

The Feistel-like structure of the ZUC S-box S_0 , consisting of P_1, P_2 and P_3 is shown in Figure 7.

A.4 Resource estimation for n -MCT in Grover's Amplification

Table 11 shows a comparison of resources required for decomposing the an n -MCT with $n = 160,288$ for amplification in Grover's (see Section 4.6) with different Toffoli reduction techniques as described in Section 2.5



(a) ZUC-128

(b) ZUC-256

A.5 Resource Estimation of ZUC $\mathcal{ENC}$ till Toffoli Level

26

Table 12: Resource comparison on NLF variants in the ZUC-256 circuit decomposed to Toffoli-level.

(a) Shallow versions.

Linear layer	S_0	depth	S_1	depth	#X	#CX	#CCX	NCT-depth	#Qubits
Yuan et al. [47]		46		103	30036	93955	26880	18082	881
Yuan et al. [47]		46		154	30528	371607	220892	20173	3001
Yuan et al. [47]		48		103	30856	93955	26880	18082	881
Yuan et al. [47]		48		154	31348	371607	220892	20173	3001
Baksi et al. [5]		46		103	30036	98957	26880	18205	991
Baksi et al. [5]		46		154	30528	376609	220892	20296	3111
Baksi et al. [5]		48		103	30856	98957	26880	18205	991
Baksi et al. [5]		48		154	31348	376609	220892	20296	3111

(b) Compact versions.

Linear layer	S_0	S_1	Toffoli reduction	#X	#CX	#CCX	NCT-depth	#Qubits
Yuan et al. [47]	S_{00}	S_{10}	Khattar et al. [27]	230526	79359	316012	186592	563
Yuan et al. [47]	S_{00}	S_{11}	Khattar et al. [27]	218636	77801	286492	168388	563
Yuan et al. [47]	S_{01}	S_{10}	Khattar et al. [27]	230526	79687	319620	186592	563
Yuan et al. [47]	S_{01}	S_{11}	Khattar et al. [27]	218636	78129	290100	168388	563
Baksi et al. [4]	S_{00}	S_{10}	Khattar et al. [27]	230526	86985	316012	186961	563
Baksi et al. [4]	S_{00}	S_{11}	Khattar et al. [27]	218636	85427	286492	168757	563
Baksi et al. [4]	S_{01}	S_{10}	Khattar et al. [27]	230526	87313	319620	186961	563
Baksi et al. [4]	S_{01}	S_{11}	Khattar et al. [27]	218636	85755	290100	168757	563
Yuan et al. [47]	S_{00}	S_{10}	Dutta et al. [14]	258406	79359	316012	142148	571
Yuan et al. [47]	S_{00}	S_{11}	Dutta et al. [14]	237660	77801	286492	127716	571
Yuan et al. [47]	S_{01}	S_{10}	Dutta et al. [14]	258406	79687	319620	142148	571
Yuan et al. [47]	S_{01}	S_{11}	Dutta et al. [14]	237660	78129	290100	127716	571
Baksi et al. [4]	S_{00}	S_{10}	Dutta et al. [14]	258406	86985	316012	142517	571
Baksi et al. [4]	S_{00}	S_{11}	Dutta et al. [14]	237660	85427	286492	128085	571
Baksi et al. [4]	S_{01}	S_{10}	Dutta et al. [14]	258406	87313	319620	142517	571
Baksi et al. [4]	S_{01}	S_{11}	Dutta et al. [14]	237660	85755	290100	128085	571
Yuan et al. [47]	S_{00}	S_{10}	Dutta et al. (optimal) [14]	30118	79359	316012	105822	571
Yuan et al. [47]	S_{00}	S_{11}	Dutta et al. (optimal) [14]	30036	77801	286492	95326	571
Yuan et al. [47]	S_{01}	S_{10}	Dutta et al. (optimal) [14]	30118	79687	319620	105822	571
Yuan et al. [47]	S_{01}	S_{11}	Dutta et al. (optimal) [14]	30036	78129	290100	95326	571
Baksi et al. [4]	S_{00}	S_{10}	Dutta et al. (optimal) [14]	30118	86985	316012	106191	571
Baksi et al. [4]	S_{00}	S_{11}	Dutta et al. (optimal) [14]	30036	85427	286492	95695	571
Baksi et al. [4]	S_{01}	S_{10}	Dutta et al. (optimal) [14]	30118	87313	319620	106191	571
Baksi et al. [4]	S_{01}	S_{11}	Dutta et al. (optimal) [14]	30036	85755	290100	95695	571

(c) Optimal versions.

Linear layer	S_0	S_1	Toffoli reduction	#X	#CX	#CCX	NCT-depth	#Qubits
Yuan et al. [47]	S_{00}	S_{10}	Khattar et al. [27]	35202	96415	45412	25954	681
Yuan et al. [47]	S_{00}	S_{11}	Khattar et al. [27]	33234	94857	45658	25667	667
Yuan et al. [47]	S_{01}	S_{10}	Khattar et al. [27]	35202	96743	49020	25954	681
Yuan et al. [47]	S_{01}	S_{11}	Khattar et al. [27]	33234	95185	49266	25667	667
Yuan et al. [47]	S_{00}	S_{10}	Dutta et al. [14]	33890	96415	45412	25626	691
Yuan et al. [47]	S_{00}	S_{11}	Dutta et al. [14]	33234	94857	45658	25667	679
Yuan et al. [47]	S_{01}	S_{10}	Dutta et al. [14]	33890	96743	49020	25626	691
Yuan et al. [47]	S_{01}	S_{11}	Dutta et al. [14]	33234	95185	49266	25667	679
Yuan et al. [47]	S_{00}	S_{10}	Dutta et al. (optimal) [14]	33890	96415	45412	24560	685
Yuan et al. [47]	S_{00}	S_{11}	Dutta et al. (optimal) [14]	33234	94857	45658	25175	671
Yuan et al. [47]	S_{01}	S_{10}	Dutta et al. (optimal) [14]	33890	96743	49020	24560	685
Yuan et al. [47]	S_{01}	S_{11}	Dutta et al. (optimal) [14]	33234	95185	49266	25175	671

A.6 Impact of MAXDEPTH Criterion

We assume that the adversary is constrained by a limit on the allowable circuit depth for executing Grover’s search. Given the substantial circuit depth required when applying Grover’s algorithm to cryptography in general, NIST introduced a constraint known as MAXDEPTH, which ranges from 2^{40} to 2^{96} . If the total circuit depth exceeds the threshold, the algorithm must be parallelised. However, as discussed in [25,37], Grover’s algorithm exhibits poor scalability under parallelisation.

Specifically, reducing the circuit depth by a factor S requires launching S^2 parallel instances of Grover’s algorithm. This results in a suboptimal trade-off between depth and resource usage. To enforce the MAXDEPTH constraint, a parallelisation strategy must be applied such that the depth is reduced to MAXDEPTH, i.e., $D/S = \text{MAXDEPTH}$, implying $S = D/\text{MAXDEPTH}$.

Under this setting, the gate count per Grover instance is reduced by a factor of S , but the number of parallel instances increases by S^2 , resulting in a total gate complexity of $G \times D/\text{MAXDEPTH}$, derived from $G/S \times S^2$. This formulation illustrates how NIST integrates gate count, circuit depth, and the MAXDEPTH constraint when evaluating the cost of quantum attacks.

For space-depth trade-off metrics such as $D-M$, where M denotes the number of qubits, the parallelization increases the qubit requirement by a factor of S^2 . Consequently, the adjusted cost becomes $\frac{D^2 \times M}{\text{MAXDEPTH}^2}$. Since the actual depth is capped at MAXDEPTH, the effective cost is expressed as $\frac{D^2 \times M}{\text{MAXDEPTH}^2}$, where D is effectively replaced by MAXDEPTH.

In summary, the $D-M$ trade-off metric under the MAXDEPTH constraint is reformulated to minimize $D^2 \times M$, indicating that optimizing circuit depth has a greater impact on resource efficiency than reducing the number of qubits. Let Td and Fd represent the T-depth and full circuit depth, respectively. Table 13 presents the comparative trade-off metrics for various quantum circuit realizations of ZUC, including Grover-parallelized settings (i.e., Td^2-M and Fd^2-M). The shallow version of the ZUC quantum circuit exhibits the best trade-off performance in terms of $Td-M$ and $Fd-M$, and also outperforms others under Grover’s parallelisation in terms of Td^2-M and Fd^2-M .

Table 13: Comparison of trade-off metrics for ZUC quantum circuits with respect to Grover’s parallelisation.

Version	Variant	Toffoli decomposition	$Td-M$	$Fd-M$	Td^2-M	Fd^2-M
Shallow	ZUC-128	Amy et al. [1]	1.03×2^{26}	1.00×2^{28}	1.22×2^{42}	1.16×2^{46}
	ZUC-128	Gidney et al. [19]	1.67×2^{24}	1.00×2^{28}	1.61×2^{39}	1.16×2^{46}
	ZUC-256	Amy et al. [1]	1.14×2^{26}	1.07×2^{28}	1.50×2^{42}	1.33×2^{46}
	ZUC-256	Gidney et al. [19]	1.85×2^{24}	1.07×2^{28}	1.98×2^{39}	1.33×2^{46}
Compact	ZUC-128	Amy et al. [1]	1.87×2^{28}	1.44×2^{30}	1.59×2^{48}	1.89×2^{51}
	ZUC-128	Gidney et al. [19]	1.20×2^{28}	1.44×2^{30}	1.32×2^{47}	1.89×2^{51}
	ZUC-256	Amy et al. [1]	1.14×2^{29}	1.59×2^{30}	1.95×2^{48}	1.15×2^{52}
	ZUC-256	Gidney et al. [19]	1.33×2^{28}	1.59×2^{30}	1.62×2^{47}	1.15×2^{52}
Optimal	ZUC-128	Amy et al. [1]	1.16×2^{26}	1.05×2^{28}	1.00×2^{43}	1.63×2^{46}
	ZUC-128	Gidney et al. [19]	1.13×2^{25}	1.05×2^{28}	1.89×2^{40}	1.63×2^{46}
	ZUC-256	Amy et al. [1]	1.28×2^{26}	1.13×2^{28}	1.22×2^{43}	1.91×2^{46}
	ZUC-256	Gidney et al. [19]	1.25×2^{25}	1.13×2^{28}	1.16×2^{41}	1.91×2^{46}